Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

CS 401/MCS 401 Lecture 3
Computer Algorithms I
Jan Verschelde, 22 June 2026

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

implementing algorithms

Niklaus Wirth: algorithms + data structures = programs.

What do we mean by *implementing an algorithm?*

Definition (1)

Implementing an algorithm is selecting the best data structures that will make the algorithm run efficiently.

Example: we proved that in the worst case, the Gale-Shapley algorithm for n employers and n applicants does require no more than n^2 iterations of the loop.

This proof does not imply that the running time of the Gale-Shapley algorithm is $O(n^2)$ because we have not proven yet that every step in each iteration of the loop has constant time.

We denote constant time by $O(1)$.

Implementing Algorithms and Graph Traversals

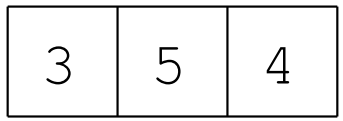
1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

arrays

An array `A` is a sequence of consecutive data elements

- of fixed size, `size(A)`, and
- the cost of accessing the i-th element in `A` is $O(1)$.

Example: an array which stores `3`, `5`, `4`:

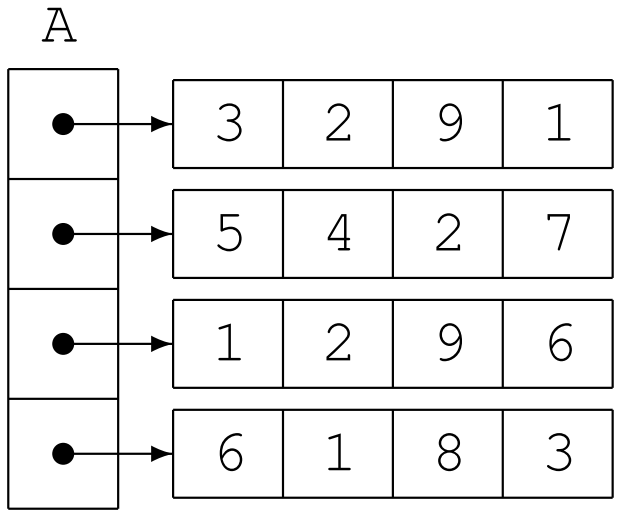


Exercise 1: Given is an array of numbers `A`, sorted in increasing order, and some number x. Prove that finding the position of x in `A` can be done in sublinear time, in the size of `A`.

two dimensional arrays

A matrix is stored as an array of arrays:

$$A = \begin{bmatrix} 3 & 2 & 9 & 1 \\ 5 & 4 & 2 & 7 \\ 1 & 2 & 9 & 6 \\ 6 & 1 & 8 & 3 \end{bmatrix}$$



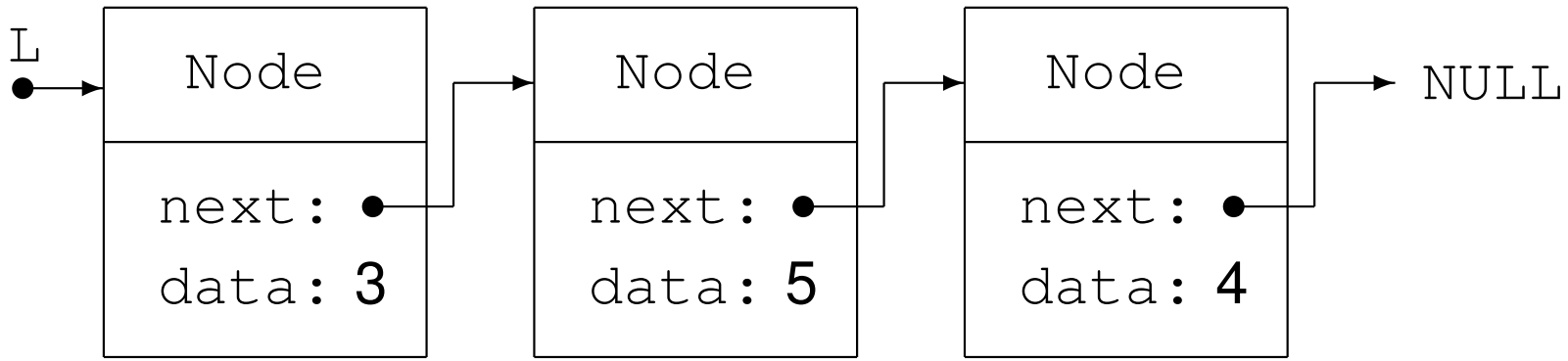
Double indexing works as follows:

$$A_{2,1} = 5 = \texttt{A}[2][1] = \texttt{A}[2,1]$$

Given indices i and j, the cost of accessing $A_{i,j} = \texttt{A}[i,j]$ is $O(1)$.

linked lists

Example: a linked list `L` to store `3`, `5`, `4`:



A node in a linked list consists of a data element and a pointer.
A linked list consists of

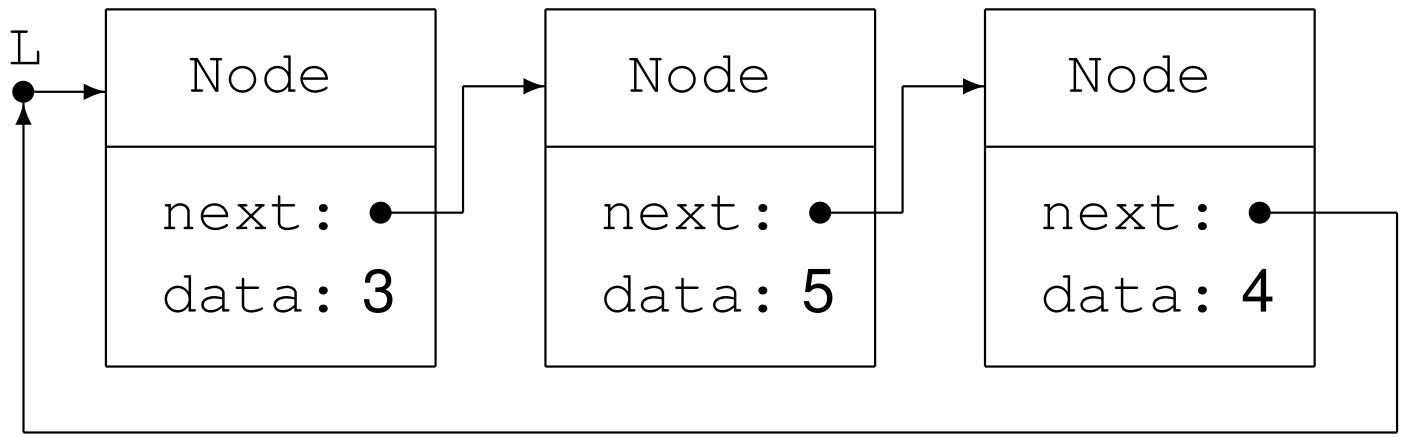
- a head node, linked to the next node,
- which is linked to the next node, etc,
- linked to the last node in the list,
  which has `NULL` as the pointer to its next node.

circular lists

In a linked list, one often needs a separate pointer to the last element of the list, to append an element to the end of the list.

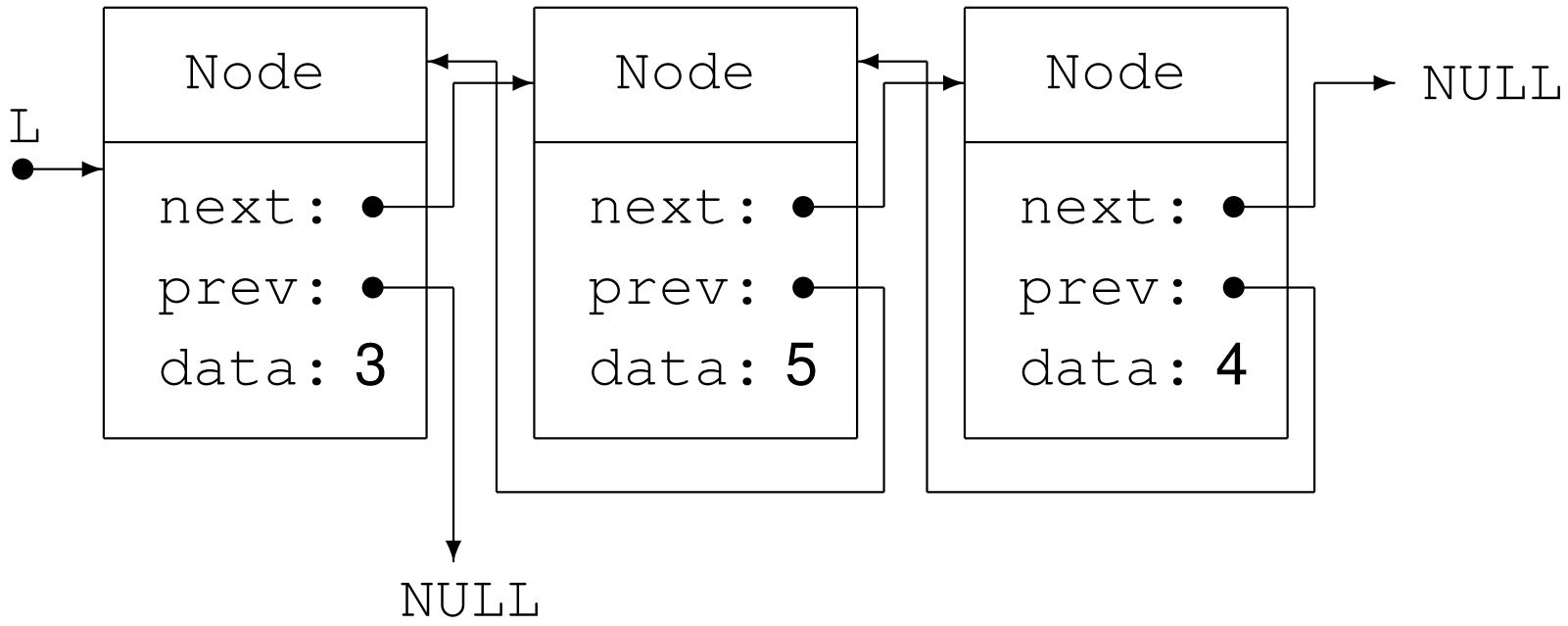
When the order among the stored data does not matter, distinguishing between first and last does not matter either, and the list is a pointer to some element in the list.

Example: a circular linked list `L` stores `3`, `5`, `4`:



doubly linked lists

Example: A doubly linked list `L` to store `3`, `5`, `4`:



At the expense of one extra pointer per node, reverse traversal becomes efficient.

Doubly linked list can also made to be circular.

properties of linked lists

The cost to locate an element in a list of n elements is $O(n)$,
because to reach the i-th element in a list,
all previous $i - 1$ nodes need to be visited.

Linked lists are dynamic data structures:

- Insertion of a new node costs $O(1)$ time.
- Given the pointer to a node in a doubly linked list,
  the deletion of the node costs $O(1)$ time.

Exercise 2: Given is a doubly linked list with n elements.

Prove the above statements: inserting and deleting a node is $O(1)$,
independent of the size n of the list.

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

input data of the Gale-Shapley algorithm

Input data: every employer and applicant has preferences with no ties.

Consider 4 employers $\{1, 2, 3, 4\}$ and 4 applicants $\{a, b, c, d\}$.

preferences for employers

1	:	a	c	b	d
2	:	b	a	d	c
3	:	a	c	b	d
4	:	d	a	c	b

preferences for applicants

a	:	2	3	4	1
b	:	1	4	2	3
c	:	1	3	4	2
d	:	2	1	3	4

Read the first line as

- employer 1 prefers a to c, prefers c to b, prefers b to d,
- applicant a prefers 2 to 3, prefers 3 to 4, prefers 4 to 1.

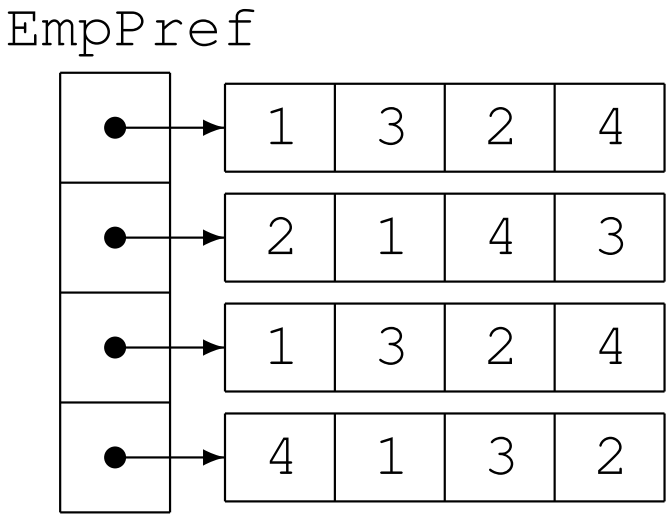
The number of employers and applicants is fixed at the start.

storing the input data

Order the n employers and n applicants into a sequence $1, 2, \ldots, n$.
Because of the fixed size of constants, we use a two dimensional array:

preferences for employers

1	:	1	3	2	4
2	:	2	1	4	3
3	:	1	3	2	4
4	:	4	1	3	2



For an employer e:
`EmpPref`$[e, i]$ stores the i-th applicant on the preference list of e.

Similarly for the applicants, the two dimensional array `AppPref` stores their preferences. For an applicant a:
`AppPref`$[a, i]$ stores the i-th employer on the preference list of a.

setup of the algorithm

Denote: E is the set of employers $e \in E$, $\#E = n$;
A is the set of applicants $a \in A$, $\#A = n$;
S is the set of committed pairs $(e, a) \in S \subset E \times A$.

Every employer and every applicant has a preference list with no ties.

Queries: `isfree`(e): there is no $a \in A$: $(e, a) \in S$;
`isfree`(a): there is no $e \in E$: $(e, a) \in S$;
`hasoffered`(e, a): e has made a job offer to a.

Initially: for all $e \in E$: `isfree`(e) is `true`,
for all $a \in A$: `isfree`(a) is `true`, and
`hasoffered`(e, a) is `false`.
$S = \emptyset$.

the Gale-Shapley algorithm

Repeat the following statements:

1. Let $e \in E$: `isfree`(e) and
   there is a $a \in A$: `hasoffered`(e, a) is `false`.

If no such a exists, then leave the repeat loop, return S.

2. Let $a \in A$: a is the highest ranked in the preference list of e
   for whom `hasoffered`(e, a) is `false`.
3. `hasoffered`$(e, a) :=$ `true`
   If `isfree`(a) then
   $S := S \cup (e, a)$ (e and a are committed)
   `isfree`$(e) :=$ `false`; `isfree`$(a) :=$ `false`
   else
   there is an $e' \in E$: $(e', a) \in S$ (a is committed to e')
   if a prefers e to e' then
   $S := S \setminus (e', a)$ (e' and a are no longer committed)
   `isfree`$(e') :=$ `true` (e' becomes free)
   $S := S \cup (e, a)$ (e and a are committed)

the first statement in the Gale-Shapley algorithm

Consider the first statement in the Gale-Shapley algorithm:

1. Let $e \in E$: `isfree`(e) and
   there is an $a \in A$: `hasoffered`(e, a) is `false`.

We need to be able to locate the next free employer.

The set of free employers changes:

- An employer is no longer free if the job offer is accepted.
- An employer becomes free if a commitment is broken.

A linked list `FreeEmp` will store the set of free employers:

- The next free employer e is at the front of the list: $O(1)$.
- If the offer is accepted, then we delete the first element: $O(1)$.
- If the applicant who accepted the offer of e breaks up with e',
  then e' is inserted to the front of the list of free employers: $O(1)$.

However, it takes $O(n)$ time to initialize `FreeEmp`.

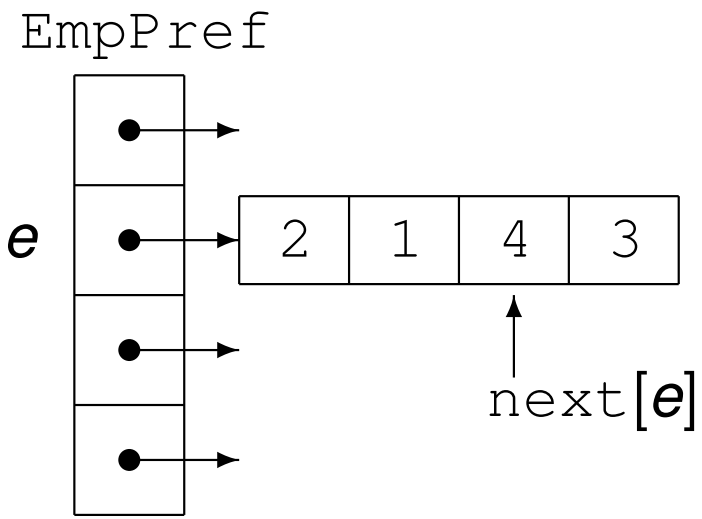
offering to the highest ranked

Consider the second statement in the Gale-Shapley algorithm:

2. Let $a \in A$: a is the highest ranked in the preference list of e for whom `hasoffered`(e, a) is `false`.

Observe:

1. Offers are made in the order of preference.
2. Preferences are stored in decreasing order: highest ranked first.



- Consider the preferences of e.
- Assume e offered to `2` and `1`.
- Next time e is free, e offers to `4`.
- `next` is an array which stores for each e the next highest ranked: e offers to `EmpPref`$[e,$ `next`$[e]]$
- Initially, `next`$[e] = 1$ for all e.

storing the commitments

Consider the beginning of the third statement in the loop:

3. $\texttt{hasoffered}(e, a) := \texttt{true}$
   If $\texttt{isfree}(a)$ then
   $S := S \cup (e, a)$

The $\texttt{hasoffered}(e, a) := \texttt{true}$ is implemented by incrementing $\texttt{next}[e]$, as: $\texttt{next}[e] := \texttt{next}[e] + 1$.

Need to know: $\texttt{isfree}(a)$ and if not, who is a committed to?

Introduce an array $\texttt{current}$ of size n:

- If $\texttt{isfree}(a)$, then $\texttt{current}[a] = 0$.
- If not $\texttt{isfree}(a)$, then $\texttt{current}[a] = e$, a is committed to e.

The $\texttt{current}$ implements the set of committed pairs S.

Because accessing an array element is $O(1)$,
checking if a is free and to whom a is committed is $O(1)$.

ranking the employers by the applicants

Consider the breaking of a commitment:

3. if ...
   else
   there is an $e' \in E$: $(e', a) \in S$ (a is committed to e')
   if a prefers e to e' then
   $S := S \setminus (e', a)$ (e' and a are no longer committed)

If there is an $e' \in E$: $(e', a) \in S$, then $\texttt{current}[a] = e'$.

How to determine *if a prefers e to e′*?
Let there be a two dimensional array `Ranking` such that

$$\texttt{Ranking}[a, e] < \texttt{Ranking}[a, e'] \text{ if } a \text{ prefers } e \text{ to } e'.$$

$\texttt{Ranking}[a, e]$ is the position of e in the preference list of a,
e.g.: if $\texttt{Ranking}[a, e] = 2$, then e is the second best choice of a.

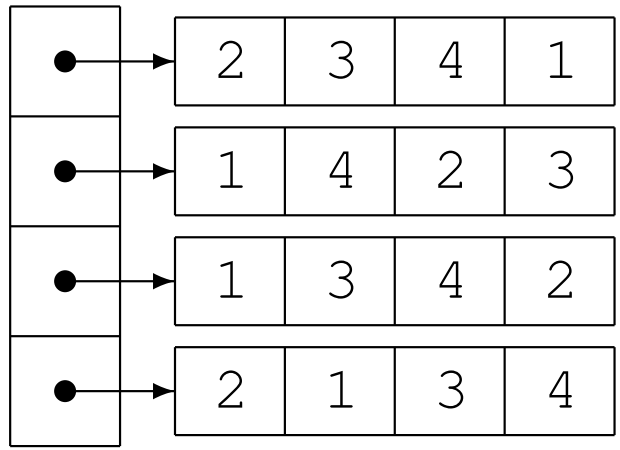
building the auxiliary data structure `Ranking`

The cost of consulting `Ranking`$[a, e]$ is $O(1)$,
but what about the cost of constructing `Ranking`?

preferences for applicants

1	:	2	3	4	1
2	:	1	4	2	3
3	:	1	3	4	2
4	:	2	1	3	4

`AppPref`



for all applicants $a = 1, 2, \ldots, n$:
  for all ranks $r = 1, 2, \ldots, n$:
    $e :=$ `AppPref`$[a, r]$
    `Ranking`$[a, e] := r$

The loop makes $2n^2$ assignments, so the cost is $O(n^2)$.

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

overview of the selected data structures

1. `EmpPref` is a two dimensional array for the preferences of n employers.
2. `AppPref` is a two dimensional array for the preferences of n applicants.
3. `FreeEmp` is a linked list of all free employers.
4. `next` is an array of size n for the next applicant to offer to.
5. `current` is an array of size n to store the committed pairs.
6. `Ranking` is a two dimensional array to rank employers.

Exercise 3: The construction of `Ranking` is $O(n^2)$. It is the most time consuming operation. In which scenario is `Ranking` not needed? Explain.

running the Gale-Shapley algorithm once again

Consider 4 employers $\{1, 2, 3, 4\}$ and 4 applicants $\{a, b, c, d\}$.

preferences for employers						preferences for applicants					
1	:	a	c	b	d	a	:	2	3	4	1
2	:	b	a	d	c	b	:	1	4	2	3
3	:	a	c	b	d	c	:	1	3	4	2
4	:	d	a	c	b	d	:	2	1	3	4

Read the first line as

- employer 1 prefers a to c, prefers c to b, prefers b to d,
- applicant a prefers 2 to 3, prefers 3 to 4, prefers 4 to 1.

Exercise 4: Run the Gale-Shapley algorithm on the above input data, show the evolution of all data structures in each step.

the Gale-Shapley algorithm is an efficient algorithm

Theorem (2)

For n employers and n applicants,
the running time of the Gale-Shapley algorithm is $O(n^2)$.

Proof. Consider the costs of the data structures defined above.

- We have two n-by-n arrays: `EmpPref` and `AppPref`, given on input. The other n-by-n array `Ranking` is constructed only once which takes $O(n^2)$ time. Accessing arrays is $O(1)$.
- The linked list `FreeEmp` takes $O(n)$ time to initialize. Deleting a free employer and inserting a free employer is $O(1)$.
- The initialization of the arrays `next` and `current` takes $O(n)$ time. Accessing arrays is $O(1)$.

After initialization, all steps in the algorithm take $O(1)$ time.
As proven in Lecture 1, the number of iterations is bounded by n^2,
so the Gale-Shapley algorithm runs in $O(n^2)$ time. Q.E.D.

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

CS 401/MCS 401 Lecture 3
Computer Algorithms I
Jan Verschelde, 22 June 2026

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

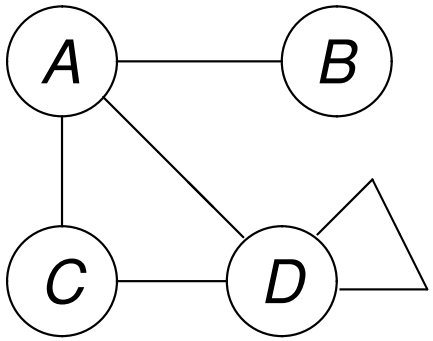
graphs, terminology and definition

- A node in a graph is called a *vertex*.
- A connection between two vertices is an *edge*.

A *graph* G is defined by a tuple (V, E):

- V is the set of vertices, and
- E is the set of edges.

An example:



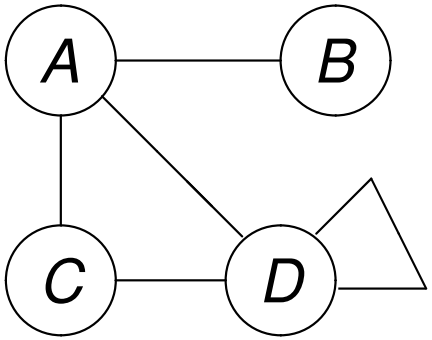
$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (A, C), (A, D), (C, D), (D, D)\}$$

Two vertices are *adjacent* if there is an edge between them.
A *path* is a sequence of successively adjacent vertices.
For example, a path from B to D is B, A, D.

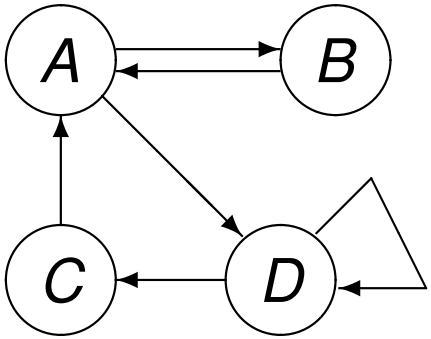
directed and undirected graphs

In an *undirected* graph (which is the default), the edge (A, B) means: there is an edge from A to B and there is an edge from B to A.



$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (A, C), (A, D), (C, D), (D, D)\}$$



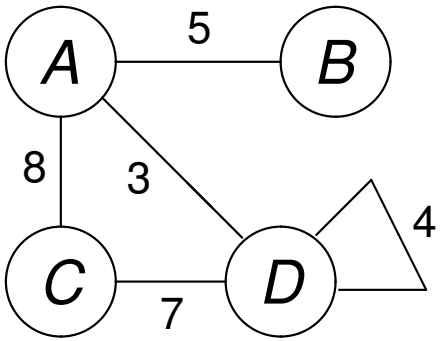
$$V = \{A, B, C, D\}$$

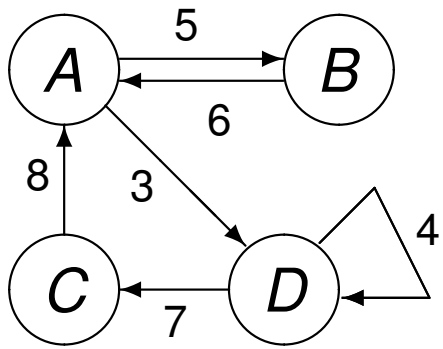
$$E = \{(A, B), (B, A), (C, A), (A, D), (D, C), (D, D)\}$$

In a *directed* graph or *digraph*, (A, B) means there is an edge from A to B and (B, A) means there is an edge from B to A.

weighted graphs

In an *weighted* graph, a value (a weight) is associated to every edge.





Edges in a weighted graphs are triplets (u, v, w),
with w the weight of the edge which connects u to v.

A *cycle* is a path where the first vertex equals the last one.
For example: A, D, C, A is a cycle.

The *degree* of a vertex is the number of edges that contain that vertex.

trees

A *tree* is an undirected, connected graph without cycles.

Theorem (3, characterization of a tree)

Let G be an undirected graph with n nodes.
G is a tree $\Leftrightarrow$ G is connected and has $n-1$ edges.

Proof by induction on n, the number of nodes.

$\Rightarrow$ G is a tree, we must show G has $n-1$ edges.

1. Base case: $n = 2$, no cycle implies one edge.
2. Induction step: Assume true for all trees with $k \leq n-1$ nodes. Consider $k = n$.

   We add one new node to the tree of $n-2$ edges, introducing one new edge.

   We obtain a connected graph with $n-1$ edges.

second part of the characterization of trees

A *tree* is an undirected, connected graph without cycles.

Theorem (3)

Let G be an undirected graph with n nodes.
G is a tree $\Leftrightarrow$ G is connected and has $n-1$ edges.

Proof by induction on n, the number of nodes.

$\Leftarrow$ G is connected and has $n-1$ edges, we must show G is a tree.

1. Base case: $n = 2$, one edge, no cycle, thus G is a tree.
2. Induction step: Assume any connected graph G with $k \leq n-2$ edges is a tree. Consider $k = n-1$.
   By the induction hypothesis, G has $n-2$ edges and no cycles.
   We add one new node by adding one new edge.
   The new edge has the new node as a vertex.
   Therefore, we do not introduce a cycle in the connected graph G, which is thus a tree. Q.E.D.

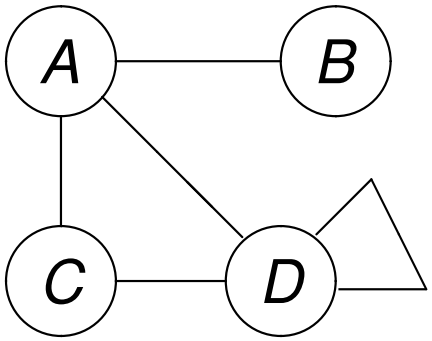
Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

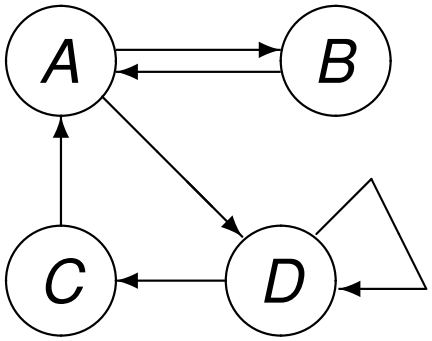
adjacency matrices

For a graph $G = (V, E)$, the *adjacency matrix* M

- has as many rows and columns as the number of vertices in V;
- for all $v_i, v_j \in V$: if $(v_i, v_j) \notin E$: $M_{i,j} = 0$;
- for all $v_i, v_j \in V$: if $(v_i, v_j) \in E$: $M_{i,j} = 1$.



	A	B	C	D
A	0	1	1	1
B	1	0	0	0
C	1	0	0	1
D	1	0	1	1

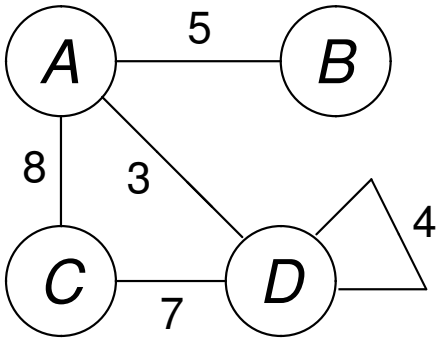


	A	B	C	D
A	0	1	0	1
B	1	0	0	0
C	1	0	0	0
D	0	0	1	1

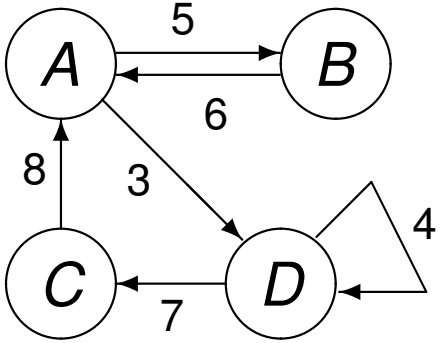
adjacency matrices for weighted graphs

For a weighted graph $G = (V, E)$, the *adjacency matrix* M

- has as many rows and columns as the number of vertices in V;
- for all $v_i, v_j \in V$: if $(v_i, v_j) \notin E$: $M_{i,j} = 0$;
- for all $v_i, v_j \in V$: if $(v_i, v_j) \in E$: $M_{i,j} = w$, w is the weight of (v_i, v_j).



	A	B	C	D
A	0	5	8	3
B	5	0	0	0
C	8	0	0	7
D	3	0	7	4



	A	B	C	D
A	0	5	0	3
B	6	0	0	0
C	8	0	0	0
D	0	0	7	4

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

finding a path in a graph

Problem: for any two vertices u and v in a graph G,
is there a path from u to v?

Consider layers in the graph $G = (V, E)$, $u \in V$:

- $L_1(u) = \left\{ v' \in V \setminus \{u\} \;\middle|\; (u, v') \in E \right\}$, all neighbors of u.
- $L_2(u) = \left\{ v' \in V \setminus \{u\} \setminus L_1(u) \;\middle|\; (v'', v') \in E, v'' \in L_1(u) \right\}$,
  all neighbors of the neighbors of u.

In general, the k-th layer, for any $k > 1$ is

$$L_k(u) = \left\{ v' \in V \setminus \{u\} \setminus \bigcup_{i=1}^{k-1} L_i(u) \;\middle|\; (v'', v') \in E, v'' \in L_{k-1}(u) \right\}.$$

There is a path from u to v if for some k: $v \in L_k(u)$.

Breadth-First Search (BFS)

Problem: given a graph $G = (V, E)$ and $u \in V$,
define a tree $T = (V_T, E_T)$, rooted at u.

1. $L_1(u) := \left\{ v' \in V \setminus \{u\} \;\middle|\; (u, v') \in E \right\}$

   $V_T := \{u\} \cup L_1(u)$; $E_T := \left\{ (u, v') \;\middle|\; v' \in L_1(u) \right\}$

   if $L_1(u) = \emptyset$, then stop, else $k := 2$ and continue to step 2

2. $L_k(u) := \left\{ v' \in V \setminus \{u\} \setminus \bigcup_{i=1}^{k-1} L_i(u) \;\middle|\; (v'', v') \in E, v'' \in L_{k-1}(u) \right\}$

   if $L_k(u) = \emptyset$, then stop

3. $V_T := V_T \cup L_k(u)$;

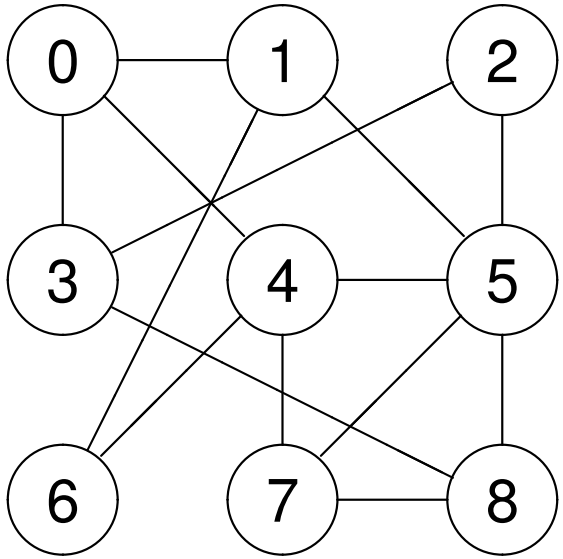
   $E_T := E_T \cup \left\{ \text{the first } (v'', v') \in E \;\middle|\; v'' \in L_{k-1}(u),\ v' \in L_k(u) \right\}$

   $k := k + 1$; go to step 2

Why is T a tree?

visiting each vertex in a graph

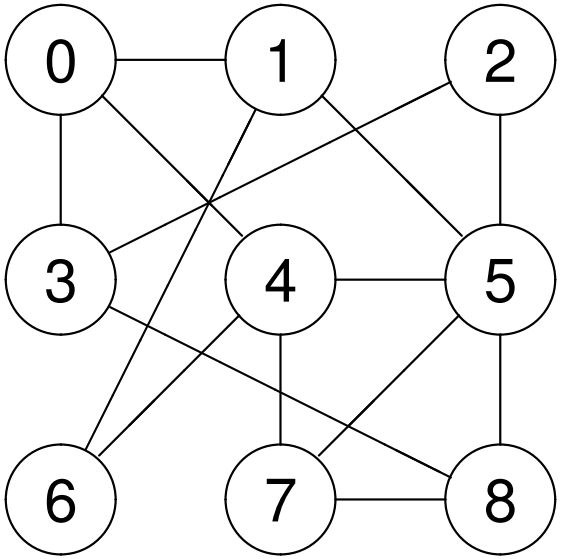
Consider the graph:

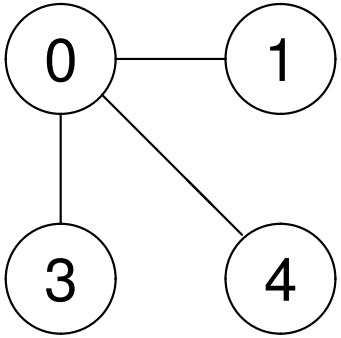


Problem: visit each vertex in a systematic order.

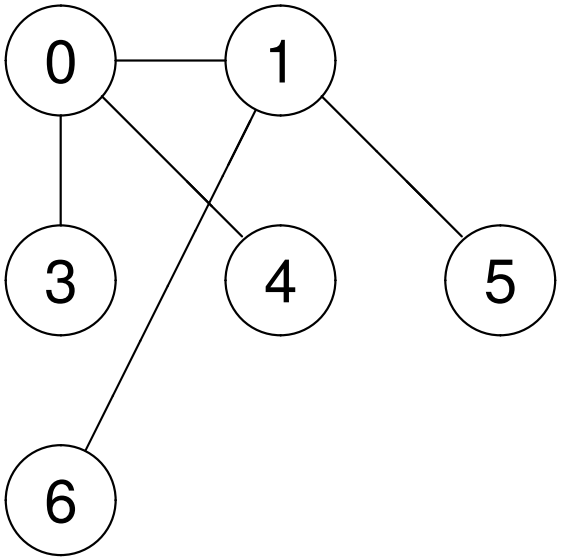
breadth-first search

We start at vertex 0.



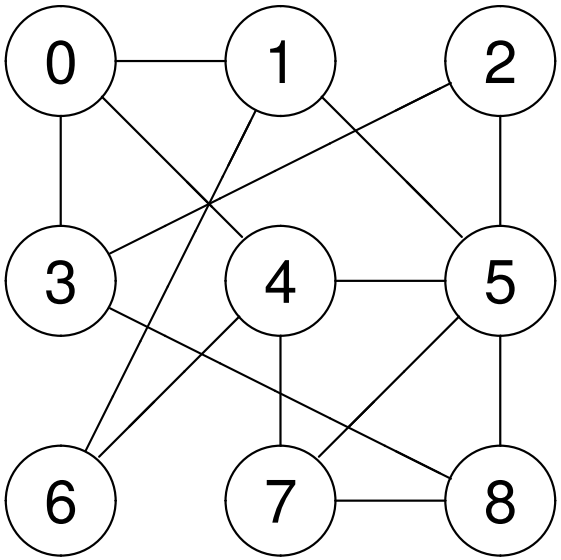


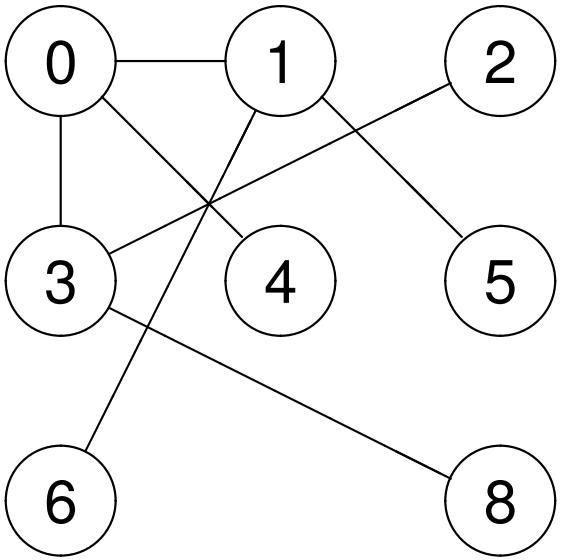
at 0: visit 1, 3, 4: $L_1 = \{1, 3, 4\}$



at 1: visit 5 and 6: $L_2 = \{5, 6\}$

breadth-first search continued



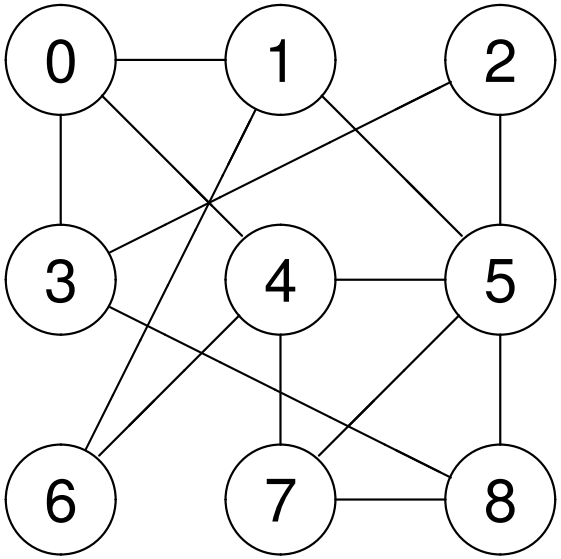


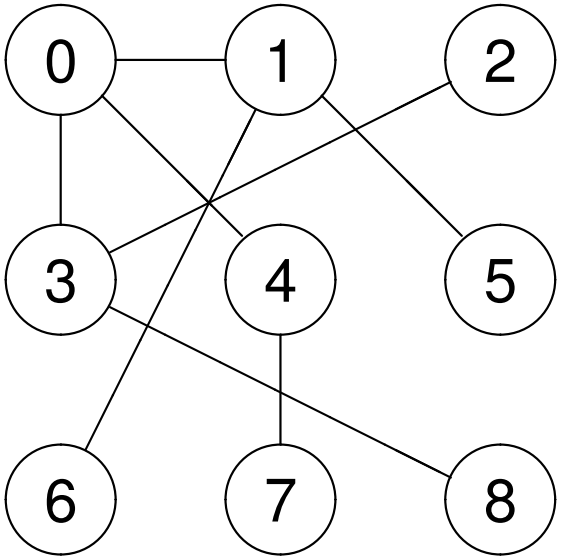
at 3: visit 2 and 8: $L_2 = L_2 \cup \{2, 8\}$

Every vertex is visited only once.

Although there is an edge from 4 to 5, the breadth-first search will not visit this edge because 5 has already been visited.

breadth-first search continued





at 4: visit 7: $L_2 = L_2 \cup \{7\}$

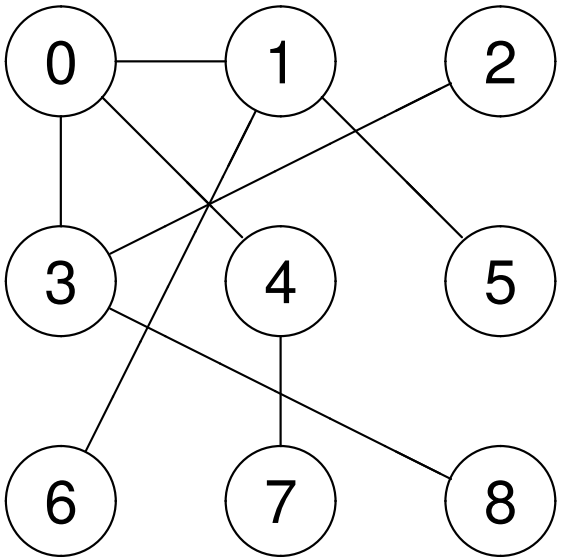
$L_1 = \{1, 3, 4\}, L_2 = \{2, 5, 6, 7, 8\}$

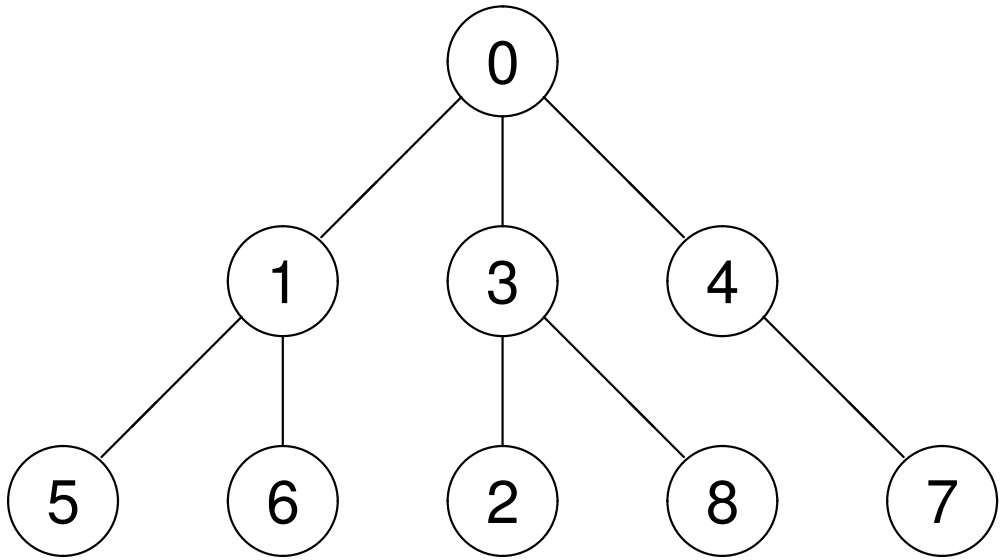
Observe that the graph on the right is a tree.

the breadth-first tree

At the left is the result of the breadth-first search.

At the right is the *breadth-first tree*.





The *breadth-first tree of a graph* contains all the vertices of the graph and the edges visited in the breadth-first search.

exploring a connected component

The BFS algorithm computes those nodes reachable from the vertex u.

Given a graph $G = (V, E)$, $u \in V$, and
the breadth-first search tree $T = (V_T, E_T)$ with root u,
the *connected component of G containing u* is
the subgraph $R = (V_R, E_R)$ of G, where

- $V_R = V_T$, and
- $E_R = \left\{ (v, w) \in E \;\middle|\; v \in V_R, w \in V_R \right\}$.

The breadth-first algorithm does not specify an order of nodes visited.

To design efficient algorithms producing search patterns with different structures, we consider depth-first search.

Implementing Algorithms and Graph Traversals

1. Data Structures
   - implementing algorithms
   - arrays and linked lists
2. Implementing the Gale-Shapley algorithm
   - selecting data structures
   - overview of the selected data structures
3. Graphs
   - definition and terminology
   - adjacency matrices
   - graph traversals: breadth-first search
   - graph traversals: depth-first search

finding the exit in a maze

Problem: for any two vertices u and v in a graph $G = (V, E)$,
is there a path from u to v?

The Depth-First Search algorithm `DFS` is formulated recursively.

For $u \in V$, the Depth-First Search algorithm `DFS` constructs the DFS tree $T = (V_T, E_T)$ recursively, starting with $u' := u$, $V_T := \emptyset$, $E_T := \emptyset$.

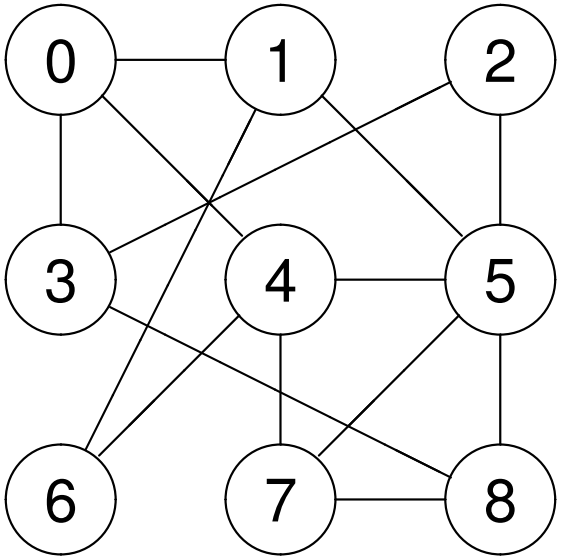
```
DFS(E, u', V_T, E_T):
    V_T := V_T ∪ {u'}                    visit u'
    for each (u', v') ∈ E                v' is connected to u'
        if v' ∉ V_T then                 if v' is not visited
            E_T := E_T ∪ {(u', v')}      add edge to tree
            DFS(E, v', V_T, E_T)         go visit v'
```

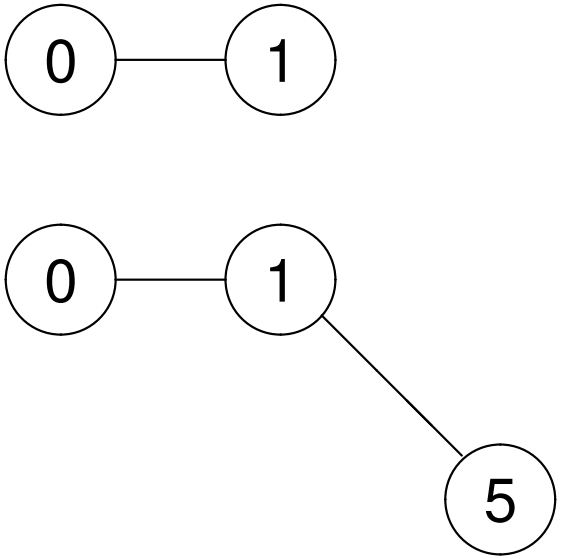
Why is T a tree?

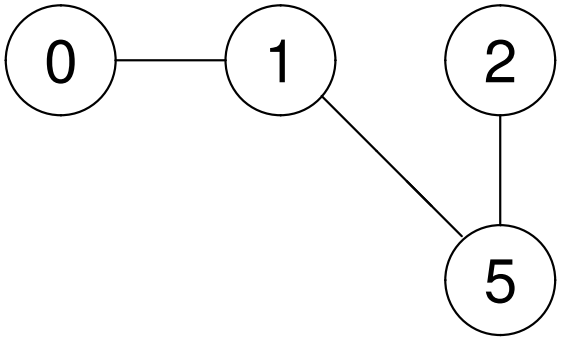
If we are interested only in reaching v, then we stop if $v' = v$.
After the if $v' \notin V_T$, `DFS` *backtracks* to the next node connected to u'.

depth-first search

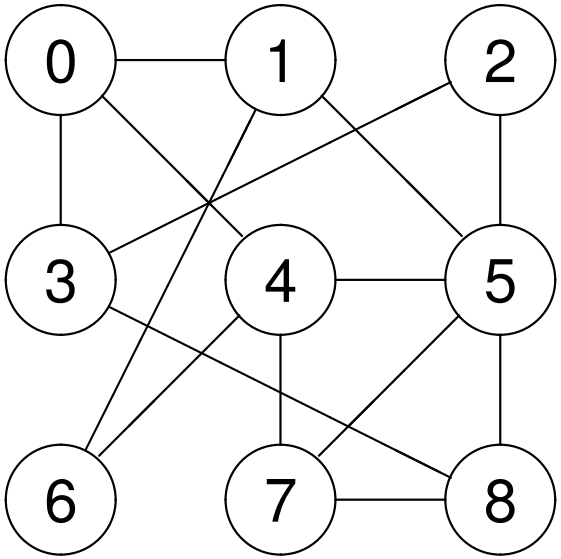
We start at vertex 0.

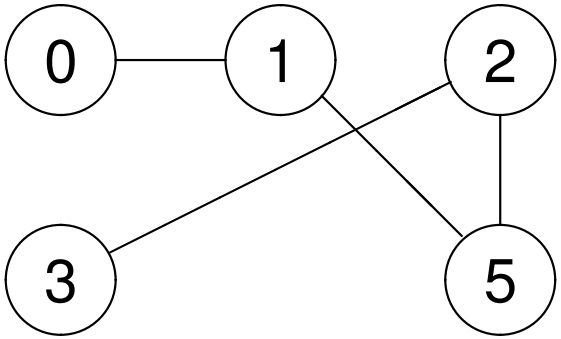


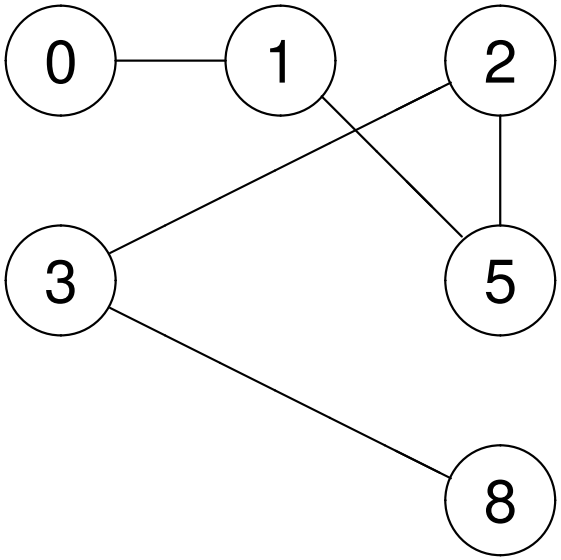




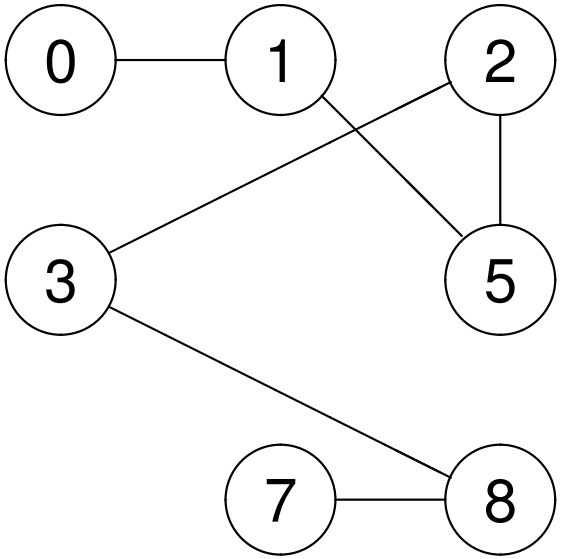
depth-first search continued

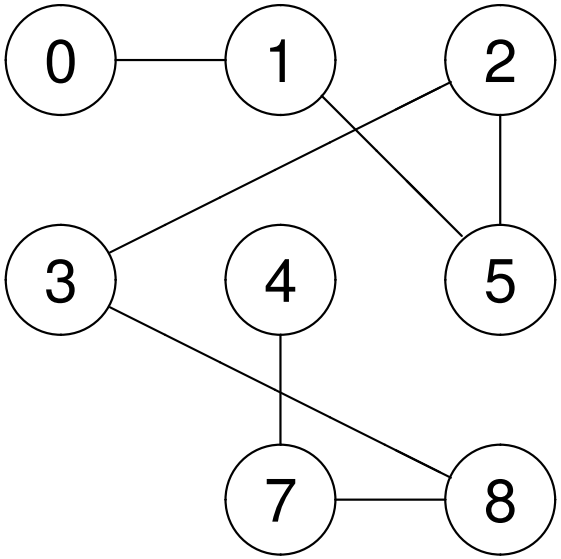




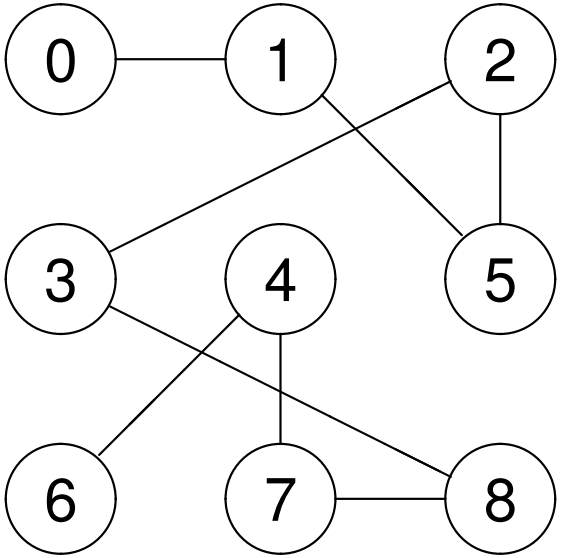


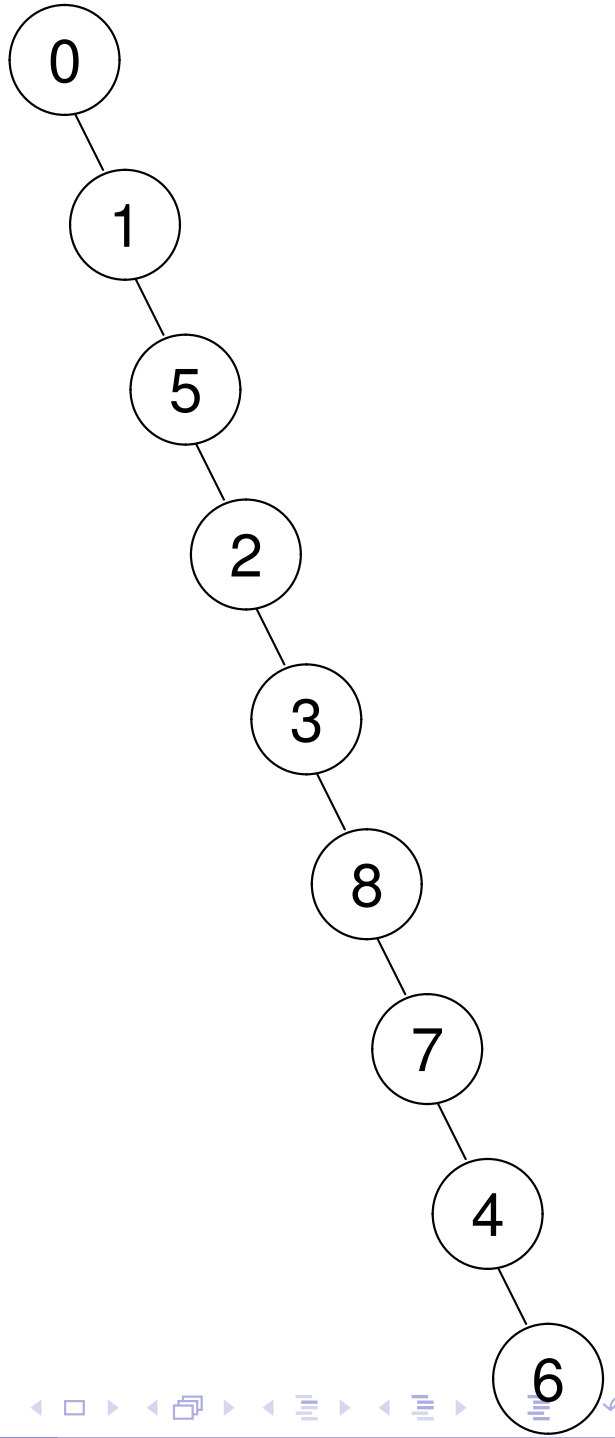
depth-first search continued





the depth-first search tree





The *depth-first search tree of a graph* contains all the vertices of the graph and the edges visited in the depth-first search.

on the shape of trees

As the example suggests:

- A BFS tree has short root-to-leaf paths.
- A DFS tree tends to be narrow and deep.

Both BFS and DFS algorithms

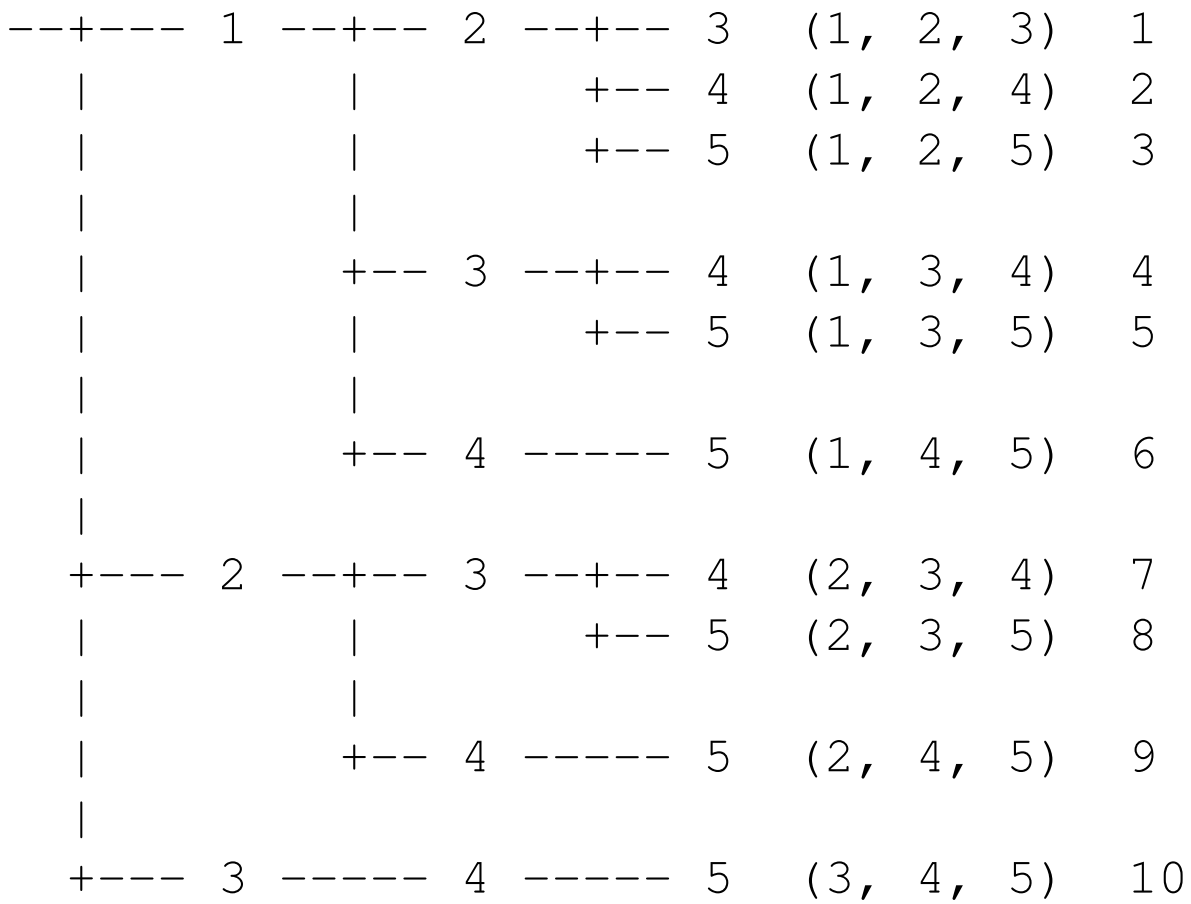
- can detect whether a graph is connected, and
- produce all connected components of a graph.

Exercise 5: When are the BFS and DFS trees the same? Justify your answer in the form of a theorem with proof.

an application: enumeration

A BFS tree occurs naturally in enumeration algorithms.

The enumeration of all subsets of three elements of $\{1, 2, 3, 4, 5\}$:



The 10 equals the 5 choose 3 number.

an enumeration algorithm

Exercise 6:

Given a set S of n elements: $S = \{1, 2, \ldots, n\}$, and a positive integer k, our problem is to enumerate all subsets of S of size k.

1. Formulate an algorithm to solve this problem.
2. Prove the correctness of your algorithm.
3. Express the number of steps as a function of n, using the big O notation.